

# Challenge 1

- 'ani\_rating.csv' 파일을 그대로 numpy 행렬로 저장하면 행의 개수는 306864개이고, 열의 개수는 571개이기 때문에, 총 **175,219,344**개의 성분을 저장하게 됨.
- 하나의 성분은 정수 값이므로 메모리에 4바이트로 저장된다고 하면 이 행렬 자체만으로도 메모리 용량이 약 **700MB**를 차지하게 됨. 관련된 행렬 연산을 할 때마다 많은 양의 메모리를 요구하게 됨으로써, 사용하고자 하는 알고리즘의 공간 복잡도(Space Complexity)가 커질수록 그 알고리즘을 수행하는 과정에서 메모리 부족 현상을 겪을 가능성이 매우 높음.
- 한편, 이 행렬의 0이 아닌 성분의 개수는 총 **2,781,900**개로, 전체 성분의 약 1.6%임. 즉, 이 행렬은 Sparse Matrix 구조를 가진다고 볼 수 있음.

# Challenge 2

- 유저와의 Correlation을 중첩된 for문으로 계산을 하게 되면 총 47,082,603,816 (470억 8260만 3816)번의 iteration이 필요함. 한 iteration에 0.001초가 걸린다고 해도, 약 1년 6개월의 시간이 소요되며, 이는 매우 비현실적인 시간임.

# Solution 1 – Sparse Matrix

- Sparse Matrix에 대한 연산은 이미 Computer Science에서 낮은 공간 복잡도를 가진 알고리즘이 제시되어 있음. scipy 라이브러리에 있는 Sparse Matrix 관련 패키지를 사용하여, 계산의 시간 및 공간 효율성을 확보함.
- 참조
  - <https://docs.scipy.org/doc/scipy/reference/sparse.html>
  - [https://en.wikipedia.org/wiki/Sparse\\_matrix](https://en.wikipedia.org/wiki/Sparse_matrix)

# Solution 2 – Vectorization

- Vectorization이란, 프로그래밍에서 array 연산을 for문 없이 효율적으로 계산하는 프로그래밍 스타일임.
- 동일한 결과를 내는 알고리즘이라도, 시간 및 공간 효율성을 위해서 Vectorization이 가능하다면 하는 것이 좋음.
- 참조
  - <https://stackoverflow.com/questions/1422149/what-is-vectorization>

# Solution 2 – Vectorization

- 예시: 주어진 두 행렬의 행벡터끼리의 Correlation 계산하기

```
# 임의의 성분을 가지는 100000x10 행렬 생성  
A = np.random.rand(100000, 10)  
B = np.random.rand(100000, 10)
```

# Solution 2 – Vectorization

## 1. Non-vectorization

각 행벡터에 대해 반복적인 작업을 수행하는 것이므로, for문을 한 번 써서, 각 행벡터마다 Correlation을 계산하고 이를 결과 행렬에 저장함.

```
C = np.zeros((100000, 1)) # 결과 행렬 (두 행렬의 각 행벡터의 Correlation을 저장)
start = time.time()
for i in range(A.shape[0]): # 각 행에 대해서
    A_mean = A[i].mean() # A의 i번째 행의 mean 계산
    B_mean = B[i].mean() # B의 i번째 행의 mean 계산
    A_square_mean = np.power(A[i], 2).mean() # A의 i번째 행의 제곱의 mean 계산
    B_square_mean = np.power(B[i], 2).mean() # B의 i번째 행의 제곱의 mean 계산
    AB_mean = np.multiply(A[i], B[i]).mean() # A의 i번째 행과 B의 i번째 행의 각 성분의 곱의 mean 계산
    cov = AB_mean - (A_mean) * (B_mean) # Covariance 계산
    var_prod = (A_square_mean - A_mean**2) * (B_square_mean - B_mean**2) # Variance 곱 계산
    corr = cov / np.sqrt(var_prod) # Correlation 계산
    C[i] = corr # 결과 Matrix에 저장
end = time.time()
```

```
print('Non-vectorization elapsed time: ' + str(end - start))
```

```
Non-vectorization elapsed time: 2.3125643730163574
```

# Solution 2 – Vectorization

아래의 과정을 행벡터 개수만큼 반복

$$A[i] = [0.6269 \quad 0.9339 \quad 0.2299 \quad 0.6539 \quad 0.4926 \quad 0.6742 \quad 0.4550 \quad 0.2817 \quad 0.4126 \quad 0.5374]$$

$$B[i] = [0.5936 \quad 0.8311 \quad 0.0325 \quad 0.3114 \quad 0.6690 \quad 0.7512 \quad 0.9891 \quad 0.6516 \quad 0.6745 \quad 0.8797]$$

$$A\_mean = np.mean(A[i]) = 0.5298$$

A의 i번째 행의 mean 계산

$$B\_mean = np.mean(B[i]) = 0.6384$$

B의 i번째 행의 mean 계산

# Solution 2 – Vectorization

```
np.power(A[i], 2) = [0.3930  0.8722  0.053  0.4276  0.2427  0.4546  0.2070  0.0794  0.1702  0.2888]  
np.power(B[i], 2) = [0.3523  0.6908  0.0010  0.0970  0.4476  0.5643  0.9783  0.4246  0.4549  0.7739]
```

```
A_square_mean = np.mean(np.power(A[i], 2)) = 0.3188
```

A의 i번째 행의 제품의 mean 계산

```
B_square_mean = np.mean(np.power(B[i], 2)) = 0.4785
```

B의 i번째 행의 제품의 mean 계산



# Solution 2 – Vectorization

```
np.multiply(A[i], B[i]) =  
[0.3721  0.7762  0.0075  0.2037  0.3296  0.5065  0.4501  0.1836  0.2783  0.4727]
```

```
AB_mean = np.multiply(A[i], B[i]).mean() = 0.3580
```

 A의 i번째 성분과 B의 i번째 성분의 곱의 mean 계산

# Solution 2 – Vectorization

$$\text{cov} = \text{AB\_mean} - (\text{A\_mean}) * (\text{B\_mean}) = 0.3580 - 0.5298 \times 0.6384 = 0.0198$$

Covariance 계산

$$\begin{aligned} \text{var\_prod} &= (\text{A\_square\_mean} - \text{A\_mean}^2) * (\text{B\_square\_mean} - \text{B\_mean}^2) \\ &= (0.3188 - 0.5298^2) \times (0.4785 - 0.6384^2) = 0.0381 \times 0.0709 = 0.0027 \end{aligned}$$

Variance 곱 계산

$$\text{corr} = \text{cov} / \text{np.sqrt}(\text{var\_prod}) = 0.0198 / \sqrt{0.0027} = 0.3805$$

Correlation 계산

# Solution 2 – Vectorization

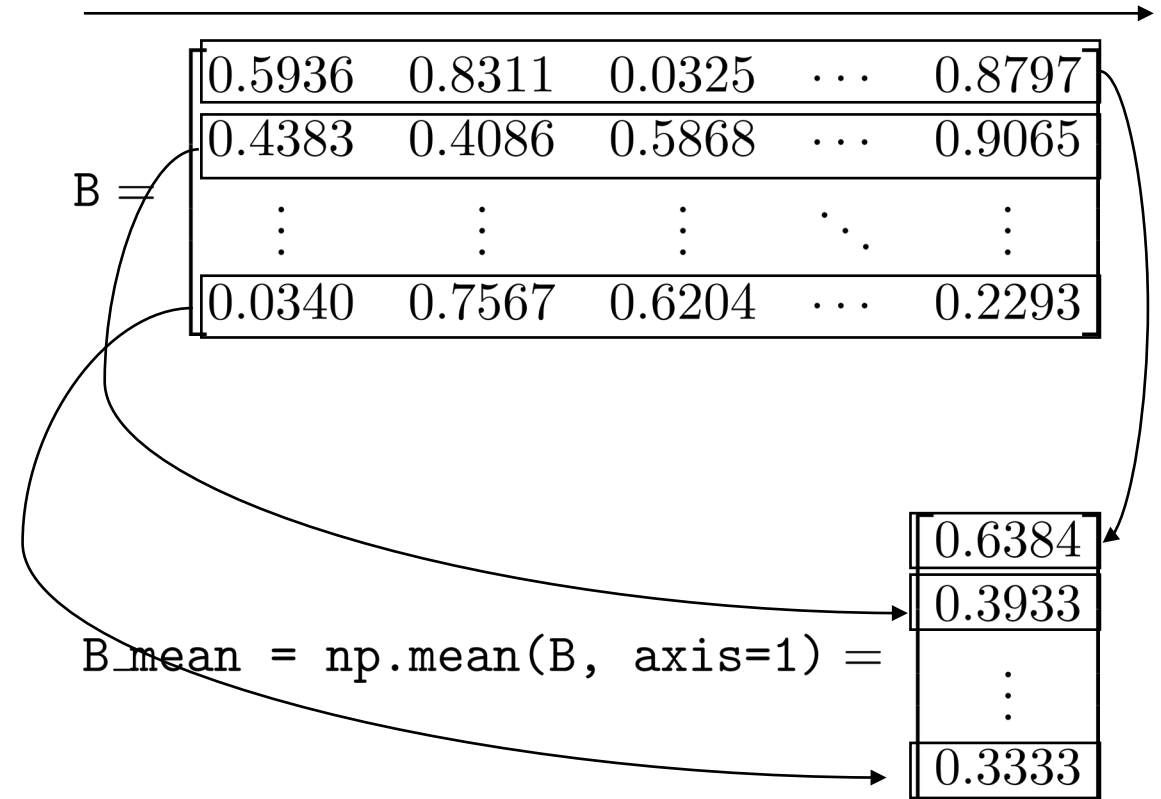
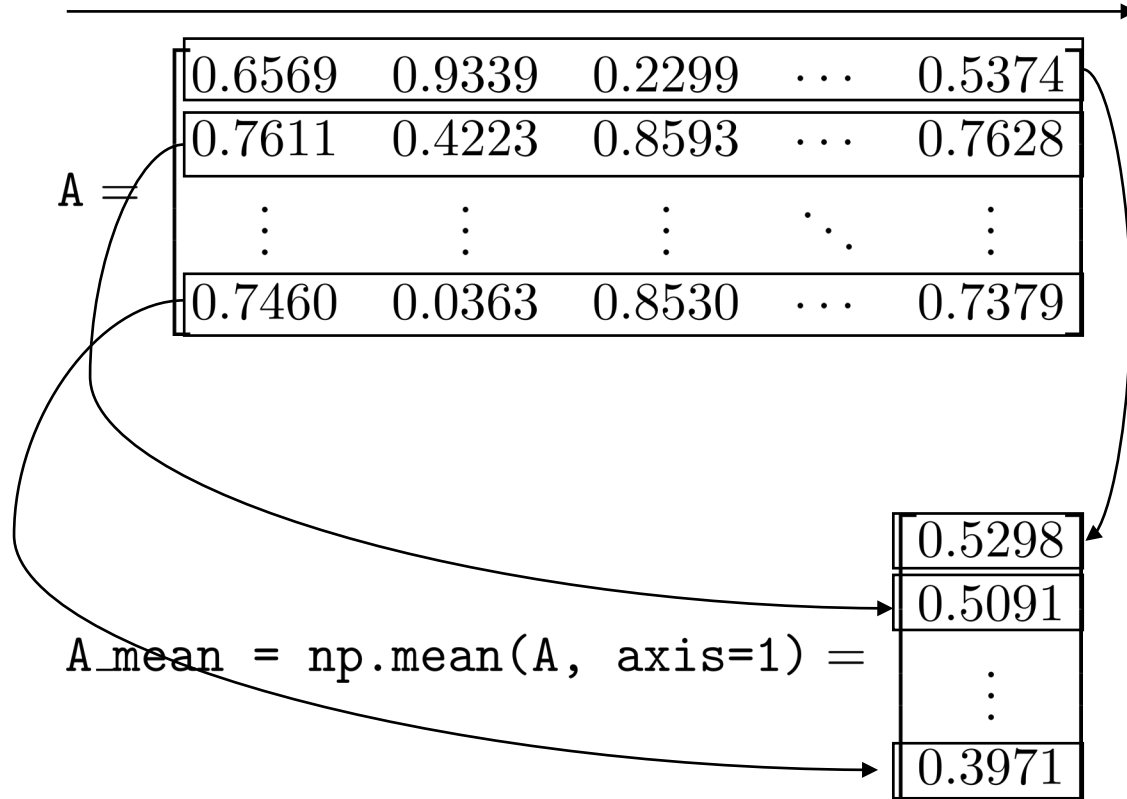
## 2. Vectorization

```
start = time.time()
A_mean = A.mean(axis=1)
B_mean = B.mean(axis=1)
A_square_mean = np.power(A, 2).mean(axis=1)
B_square_mean = np.power(B, 2).mean(axis=1)
AB_mean = np.multiply(A, B).mean(axis=1)
cov = AB_mean - np.multiply(A_mean, B_mean)
var_prod = np.multiply(A_square_mean - A_mean**2, B_square_mean - B_mean**2)
C_vec = np.divide(cov, np.sqrt(var_prod)).reshape(-1, 1)
end = time.time()
```

```
print('Vectorization elapsed time: ' + str(end - start))
```

```
Vectorization elapsed time: 0.04504251480102539
```

# Solution 2 – Vectorization



# Solution 2 – Vectorization

`A_square = np.power(A, 2) =`

|        |        |        |     |        |
|--------|--------|--------|-----|--------|
| 0.3930 | 0.8722 | 0.0528 | ... | 0.2888 |
| 0.5793 | 0.1783 | 0.7384 | ... | 0.5819 |
| ⋮      | ⋮      | ⋮      | ⋱   | ⋮      |
| 0.5565 | 0.0013 | 0.7277 | ... | 0.5444 |

`A_square_mean = np.mean(np.power(A, 2), axis=1) =`

|        |
|--------|
| 0.3188 |
| 0.3237 |
| ⋮      |
| 0.2623 |

# Solution 2 – Vectorization

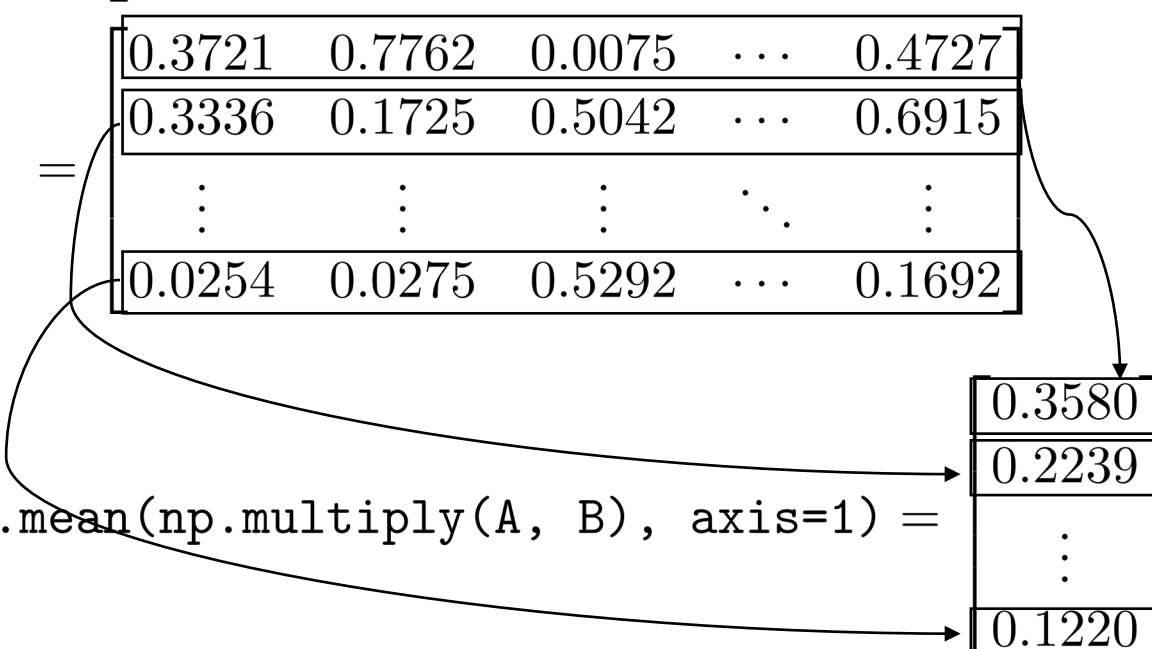
`B_square = np.power(B, 2) =`

|        |        |        |     |        |
|--------|--------|--------|-----|--------|
| 0.3523 | 0.6908 | 0.0011 | ... | 0.7739 |
| 0.1921 | 0.1670 | 0.3443 | ... | 0.8218 |
| ⋮      | ⋮      | ⋮      | ⋱   | ⋮      |
| 0.0012 | 0.5726 | 0.3849 | ... | 0.0526 |

`B_square_mean = np.mean(np.power(B, 2), axis=1) =`

|        |
|--------|
| 0.4785 |
| 0.2177 |
| ⋮      |
| 0.1612 |

# Solution 2 – Vectorization

$$\begin{aligned} \text{AB} = \text{np.multiply}(\text{A}, \text{B}) &= \begin{bmatrix} 0.6269 \times 0.5936 & 0.9334 \times 0.8311 & 0.2299 \times 0.0325 & \dots & 0.5374 \times 0.8797 \\ 0.7611 \times 0.4383 & 0.4223 \times 0.4086 & 0.8593 \times 0.5868 & \dots & 0.7628 \times 0.9065 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0.7460 \times 0.0340 & 0.0363 \times 0.7567 & 0.8530 \times 0.6204 & \dots & 0.7379 \times 0.2293 \end{bmatrix} \\ &= \begin{bmatrix} 0.3721 & 0.7762 & 0.0075 & \dots & 0.4727 \\ 0.3336 & 0.1725 & 0.5042 & \dots & 0.6915 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0.0254 & 0.0275 & 0.5292 & \dots & 0.1692 \end{bmatrix} \\ \text{AB\_mean} = \text{np.mean}(\text{np.multiply}(\text{A}, \text{B}), \text{axis}=1) &= \begin{bmatrix} 0.3580 \\ 0.2239 \\ \vdots \\ 0.1220 \end{bmatrix} \end{aligned}$$


# Solution 2 – Vectorization

$$\begin{aligned} \text{cov} = \text{AB\_mean} - \text{np.multiply}(\text{A\_mean}, \text{B\_mean}) &= \begin{bmatrix} 0.3580 \\ 0.2239 \\ \vdots \\ 0.1220 \end{bmatrix} - \begin{bmatrix} 0.3382 \\ 0.2002 \\ \vdots \\ 0.1323 \end{bmatrix} \\ &= \begin{bmatrix} 0.0198 \\ 0.0236 \\ \vdots \\ -0.0104 \end{bmatrix} \end{aligned}$$



# Solution 2 – Vectorization

$$A\_var = A\_square\_mean - A\_mean**2 = \begin{bmatrix} 0.3188 \\ 0.3237 \\ \vdots \\ 0.2623 \end{bmatrix} - \begin{bmatrix} 0.2807 \\ 0.2592 \\ \vdots \\ 0.1577 \end{bmatrix} = \begin{bmatrix} 0.0381 \\ 0.0645 \\ \vdots \\ 0.1046 \end{bmatrix}$$

$$B\_var = B\_square\_mean - B\_mean**2 = \begin{bmatrix} 0.4785 \\ 0.2177 \\ \vdots \\ 0.1611 \end{bmatrix} - \begin{bmatrix} 0.4075 \\ 0.1547 \\ \vdots \\ 0.1111 \end{bmatrix} = \begin{bmatrix} 0.0710 \\ 0.0630 \\ \vdots \\ 0.0501 \end{bmatrix}$$

```
var_prod = np.multiply(A_square_mean - A_mean**2, B_square_mean - B_mean**2)
```

$$= \begin{bmatrix} 0.0027 \\ 0.0041 \\ \vdots \\ 0.0052 \end{bmatrix}$$

# Solution 2 – Vectorization

```
C_vec = np.divide(cov, var_prod).reshape(-1, 1)
```

$$= \begin{bmatrix} 0.0198/0.0027 \\ 0.0236/0.0041 \\ \vdots \\ -0.0104/0.0052 \end{bmatrix} = \begin{bmatrix} 0.3805 \\ 0.3709 \\ \vdots \\ -0.1432 \end{bmatrix}$$

# Solution 2 – Vectorization

C

```
array([[ 0.38053604],  
       [ 0.37088325],  
       [-0.21801677],  
       ...,  
       [ 0.45307519],  
       [-0.34159347],  
       [-0.14318309]])
```

C\_vec

```
array([[ 0.38053604],  
       [ 0.37088325],  
       [-0.21801677],  
       ...,  
       [ 0.45307519],  
       [-0.34159347],  
       [-0.14318309]])
```

같은 결과가 나오지만, Vectorization이 약 **50여배** 빠름.

```
raw_mat = ani_rating_df[[col for col in ani_rating_df.columns if col != 'svc_mgmt_num']].copy(True).to_numpy()
CF_mat_one = sp.sparse.csr_matrix(np.where((raw_mat != 1) & (raw_mat != 0), 1, raw_mat))
CF_mat = sp.sparse.csr_matrix(raw_mat)
CS_mat = ani_rating_df['svc_mgmt_num'].to_numpy()
```

1. 'ani\_rating.csv' 파일을 pandas로 불러와 DataFrame으로 저장하고, 이를 to\_numpy() 함수를 이용하여 raw\_mat이라는 이름의 numpy array로 저장.
2. raw\_mat에서 특정 위치의 성분이 0인지 아닌지를 나타내는 행렬인 CF\_mat\_one을 scipy의 csr matrix로 저장.
3. numpy array raw\_mat을 scipy의 csr matrix로 저장.
4. 고객 ID를 성분으로 갖는 numpy array를 CS\_mat이라는 이름으로 저장

```
raw_mat = ani_rating_df[[col for col in ani_rating_df.columns if col != 'svc_mgmt_num']].copy(True).to_numpy()
CF_mat_one = sp.sparse.csr_matrix(np.where((raw_mat != 1) & (raw_mat != 0), 1, raw_mat))
CF_mat = sp.sparse.csr_matrix(raw_mat)
CS_mat = ani_rating_df['svc_mgmt_num'].to_numpy()
```

- \* CF\_mat의 행은 각 유저를 나타내고, 열은 애니메이션을 나타냄.
- \* CS\_mat의 행은 CF\_mat의 각 행에 대응하는 유저의 ID를 나타냄.
- \* CF\_mat\_one은 기존의 CF\_mat 성분을 모두 1로 바꾼 것임.

```
positive_rating_idx_dict = {} # 'ani_col_idx: list of index'
for col in range(CF_mat.shape[1]):
    positive_rating_idx_dict[col] = sp.sparse.find(CF_mat[:, col] > 0)
```

각 애니메이션마다 시청한 유저들의 인덱스를 '애니메이션 칼럼 인덱스: 유저 인덱스 리스트' 형식의 dictionary에 저장

# Code 설명 – get\_cf\_feasibility

```
1     ret = {}  
  
2     [ this_CF_mat = CF_mat[i]  
      this_CF_mat_one = CF_mat_one[i]  
      this_user_id = CS_mat[i]  
3     fail = False
```

1. 반환할 dictionary를 ret로 정의.
2. CF\_mat, CF\_mat\_one, CS\_mat으로 부터, 각각 인덱스 i에 대응하는 행벡터를 this\_CF\_mat, this\_CF\_mat\_one, this\_user\_id로 저장.
3. 초기 CF로 추천 가능 여부 변수를 fail로 이름짓고, 초기값 False로 할당.

# Code 설명 – get\_cf\_feasibility

```
    # 해당 유저의 Implicit Rating의 분산이 0일 경우 계산 불가
4   this_user_var = var_sparse_mat(this_CF_mat, axis=1)
    if this_user_var == 0:
        fail = True
        num_common_user = np.nan
5   avg_common_ani_num = np.nan
        std_common_ani_num = np.nan
        pair_string = ""
        cf_fail_reason = "No variance in implicit rating"
```

4. this\_CF\_mat의 분산을 계산하고, 이를 this\_user\_var에 저장.

5. 만약 분산이 0이라면, fail을 True로 할당하고, 관련 통계량을 nan으로 할당. 실패 이유는 “Implicit Rating에 Variance가 없음”으로 저장.



# Code 설명 – get\_cf\_feasibility

```
else:
6  this_user_mat = CF_mat.multiply(this_CF_mat_one)
7  this_user_mat_one = CF_mat_one.multiply(this_CF_mat_one)
8  this_user_vec = np.asarray(this_user_mat_one.sum(axis=1))
   if std or export_mode:
       this_rating_mean = np.asarray(mean_sparse_mat(this_user_mat[i], axis=1)).reshape(-1).item()
       this_rating_std = np.asarray(np.sqrt(var_sparse_mat(this_user_mat[i], axis=1))).reshape(-1).item()
```

6. 다른 유저의 rating에 대해, 현재 유저가 시청한 애니메이션에 대한 rating만 남겨두고, 나머지 성분들은 0으로 만듦. 이를 `this_user_mat`에 저장.

7. 다른 유저의 Rating에 대해, 현재 유저와 시청한 애니메이션인 경우 1, 아닐 경우 0의 성분을 가지는 행렬을 `this_user_mat_one`으로 저장.

8. 현재 유저와 시청한 애니메이션 중 겹치는 애니메이션의 개수를 성분으로 가지는 벡터를 `this_user_vec`으로 저장.

# Code 설명 – get\_cf\_feasibility

```
this_CF_mat = CF_mat[i] = [0 0 4 4 5 1 0]
```

```
this_CF_mat_one = CF_mat_one[i] = [0 0 1 1 1 1 0]
```

`CF_mat.multiply(this_CF_mat_one)` = 
$$\begin{bmatrix} 3 & 4 & 0 & 1 & 0 & 2 & 3 \\ 1 & 0 & 1 & 2 & 5 & 5 & 3 \\ 0 & 0 & 4 & 4 & 5 & 1 & 0 \\ 0 & 2 & 0 & 1 & 3 & 5 & 4 \end{bmatrix} \cdot \text{multiply}([0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0])$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 2 & 0 \\ 0 & 0 & 1 & 2 & 5 & 5 & 0 \\ 0 & 0 & 4 & 4 & 5 & 1 & 0 \\ 0 & 0 & 0 & 1 & 3 & 5 & 0 \end{bmatrix}$$

`this_user_mat` ←

해당 유저가 시청한 애니메이션에 대한 평점만 남겨놓음.

성분끼리 곱하는 함수에서 두 행렬의 사이즈가 다를 경우, Broadcasting이 일어남. 여기서는 오른쪽 행렬의 사이즈가 왼쪽 행렬 사이즈에 맞게 아래와 같이 변환됨.

$$\begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

# Code 설명 – get\_cf\_feasibility

```
this_CF_mat = CF_mat[i] = [0 0 4 4 5 1 0]
```

```
this_CF_mat_one = CF_mat_one[i] = [0 0 1 1 1 1 0]
```

`CF_mat_one.multiply(this_CF_mat_one)` = 
$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \cdot \text{multiply}([0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0])$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

`this_user_mat_one`

해당 유저가 시청한 애니메이션에 대한 시청 여부만 남겨놓음.

# Code 설명 – get\_cf\_feasibility

```
this_CF_mat = CF_mat[i] = [0  0  4  4  5  1  0]
```

```
this_CF_mat_one = CF_mat_one[i] = [0  0  1  1  1  1  0]
```

```
this_user_vec =  $\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \cdot \text{sum}(\text{axis}=1) = \begin{bmatrix} 2 \\ 4 \\ 4 \\ 3 \end{bmatrix}$ 
```

해당 유저와 시청한 애니메이션이 겹치는 수

# Code 설명 – get\_cf\_feasibility

```
else:
    this_user_mat = CF_mat.multiply(this_CF_mat_one)
    this_user_mat_one = CF_mat_one.multiply(this_CF_mat_one)
    this_user_vec = np.asarray(this_user_mat_one.sum(axis=1))
    if std or export_mode:
        9 this_rating_mean = np.asarray(mean_sparse_mat(this_user_mat[i], axis=1)).reshape(-1).item()
        this_rating_std = np.asarray(np.sqrt(var_sparse_mat(this_user_mat[i], axis=1))).reshape(-1).item()
```

평점을 표준화해서 계산할 경우,

9. 현재 유저의 평점의 평균과 표준편차를 각각 `this_rating_mean`과 `this_rating_std`에 저장

# Code 설명 – get\_cf\_feasibility

```
# 겹치지 않는 유저들은 삭제
no_overlap_user_idx = np.append(np.where(this_user_vec == 0)[0], i)
this_user_mat = delete_rows_csr(this_user_mat, no_overlap_user_idx)
10 this_user_mat_one = delete_rows_csr(this_user_mat_one, no_overlap_user_idx)
    this_user_vec = np.delete(this_user_vec, no_overlap_user_idx)
11 this_user_num = this_user_mat.shape[0]
12 this_other_user_id_mat = np.delete(CS_mat, no_overlap_user_idx)
```

10. 해당 유저와 시청한 애니메이션이 아예 겹치지 않는 유저들 및 해당 유저 자체의 기록을 삭제.

11. 삭제 후 남은 유저 수 (즉, 해당 유저를 제외하고 적어도 하나는 시청한 애니메이션이 겹치는 유저 수)를 `this_user_num`에 저장.

12. 삭제 후 남은 유저 ID를 `this_user_id_mat`라는 이름의 numpy array로 저장.

# Code 설명 – get\_cf\_feasibility

```
# 조건 1. 시청한 애니메이션이 겹치는 다른 유저들 중 최소 한 유저는 평점 분산이 0보다 크다
13 var_mat = var_sparse_mat(this_user_mat, axis=1)
14 filter_idx = sp.sparse.find((var_mat == 0) | (var_mat == np.nan))[0]
    if len(filter_idx) == this_user_num:
        fail = True
        num_common_user = np.nan
        avg_common_ani_num = np.nan
15     std_common_ani_num = np.nan
        pair_string = ""
        cf_fail_reason = "No variance in implicit ratings of the others who overlap in watched animations"
```

13. 다른 유저들의 해당 유저가 시청한 애니메이션에 대한 평점 분산 계산.

14. 분산이 0이거나 정의되지 않는 유저들의 인덱스를 찾음.

15. 만약 모든 유저들의 평점 분산이 올바르지 않다면, fail을 True로 할당하고, 관련 통계량을 nan값으로 할당. 실패 이유는 “겹치는 애니메이션에 대한 평점의 분산이 없음”으로 저장.

# Code 설명 – get\_cf\_feasibility

```
16 # 조건 2. 다른 유저들의 겹치는 애니메이션의 평점의 분산이 0보다 크다
17 this_user_mat_overlap = this_user_mat_one.multiply(CF_mat[i]) # 대상 유저와 겹치는 애니메이션 평점만 남김
18 filter_idx = np.union1d(filter_idx, sp.sparse.find(var_sparse_mat(this_user_mat_overlap, axis=1) == 0)[0])
19 if len(filter_idx) == this_user_num:
20     fail = True
21     num_common_user = np.nan
22     avg_common_anime_num = np.nan
23     std_common_anime_num = np.nan
24     pair_string = ""
25     cf_fail_reason = "No variance in implicit ratings of the others who overlap in watched animations"
```

16. 다른 유저들 중 해당 유저와 겹치는 애니메이션에 대한 해당 유저의 분산 계산

17. 분산이 0이거나 정의되지 않는 유저들의 인덱스를 찾음.

18. 만약 다른 모든 유저들에 대해 해당 유저의 평점 분산이 올바르게 없다면, fail을 True로 할당하고, 관련 통계량을 nan값으로 할당. 실패 이유는 “겹치는 애니메이션에 대한 평점의 분산이 없음”으로 저장.



조건 1과 조건 2의 차이:

해당 유저를 A라고 하고, 다른 특정 유저를 B라고 하면,

A가 시청한 애니메이션들에 대해 **B의 분산**을 확인하는 것이 조건 1임. 이때, B는 A가 시청한 애니메이션을 시청하지 않았을 수도 있고, 그런 애니메이션은 제외하고 B의 분산을 확인함.

반면 조건 2는 A와 B가 모두 시청한 애니메이션에 대해서, **A의 분산**을 확인함. 비록 A는 여러 애니메이션에 대해 다양한 평점을 남겼을 수도 있지만, correlation을 계산하고자 하는 B와 겹치는 애니메이션에 대해서는 일관된 평점을 남겼을 수도 있음.

# Code 설명 – get\_cf\_feasibility

```
this_CF_mat = CF_mat[i] = [0 0 4 4 5 1 0]
```

```
this_CF_mat_one = CF_mat_one[i] = [0 0 1 1 1 1 0]
```

```
this_user_mat_one.multiply(CF_mat[i]) =  $\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \cdot \text{multiply}([0 \ 0 \ 4 \ 4 \ 5 \ 1 \ 0])$ 
```

```
=  $\begin{bmatrix} 0 & 0 & 0 & 4 & 0 & 1 & 0 \\ 0 & 0 & 4 & 4 & 5 & 1 & 0 \\ 0 & 0 & 4 & 4 & 5 & 1 & 0 \\ 0 & 0 & 0 & 4 & 5 & 1 & 0 \end{bmatrix}$ 
```

`this_user_mat_overlap`

←  
각 행 벡터의 분산은 해당 유저와 특정 다른 유저와 시청한 애니메이션 중 겹치는 애니메이션에 대한 해당 유저의 평점 분산임

# Code 설명 – get\_cf\_feasibility

```
19      # 조건 3. 겹치는 애니메이션의 수가 최소 정한 수치 (min_overlap) 이상이어야 한다
      filter_idx = np.union1d(filter_idx, np.where(this_user_vec < min_overlap)[0])
      if len(filter_idx) == this_user_num:
20         fail = True
         num_common_user = np.nan
         avg_common_anime_num = np.nan
         std_common_anime_num = np.nan
         pair_string = ""
         cf_fail_reason = "Not enough the number of others who overlap in watched animations"
```

19. 해당 유저와 겹치는 애니메이션의 수가 정해진 수치(min\_overlap)보다 작은 유저들의 인덱스를 찾음.

20. 만약 다른 모든 유저들이 정해진 수치보다 겹치는 애니메이션 수가 작다면, fail을 True로 할당하고, 관련 통계량을 nan값으로 할당. 실패 이유는 “겹치는 애니메이션 수 부족”으로 저장.

# Code 설명 – get\_cf\_feasibility

```
21     ret['SUMMARY_STAT'] = {}  
      # 조건에 맞지 않은 유저들 및 해당 유저는 모두 계산되는 행렬에서 삭제  
      this_user_mat = delete_rows_csr(this_user_mat, filter_idx)  
      this_user_mat_overlap = delete_rows_csr(this_user_mat_overlap, filter_idx)  
22     this_user_mat_one = delete_rows_csr(this_user_mat_one, filter_idx)  
      this_user_vec = np.delete(this_user_vec, filter_idx).reshape(-1)  
      this_other_user_id_mat = np.delete(this_other_user_id_mat, filter_idx)
```

21. ret에 'SUMMARY\_STAT' 키의 값을 dictionary로 지정

22. 조건1, 조건2, 조건3 중 하나라도 맞지 않는 유저들은 모두 계산되는 행렬에서 삭제

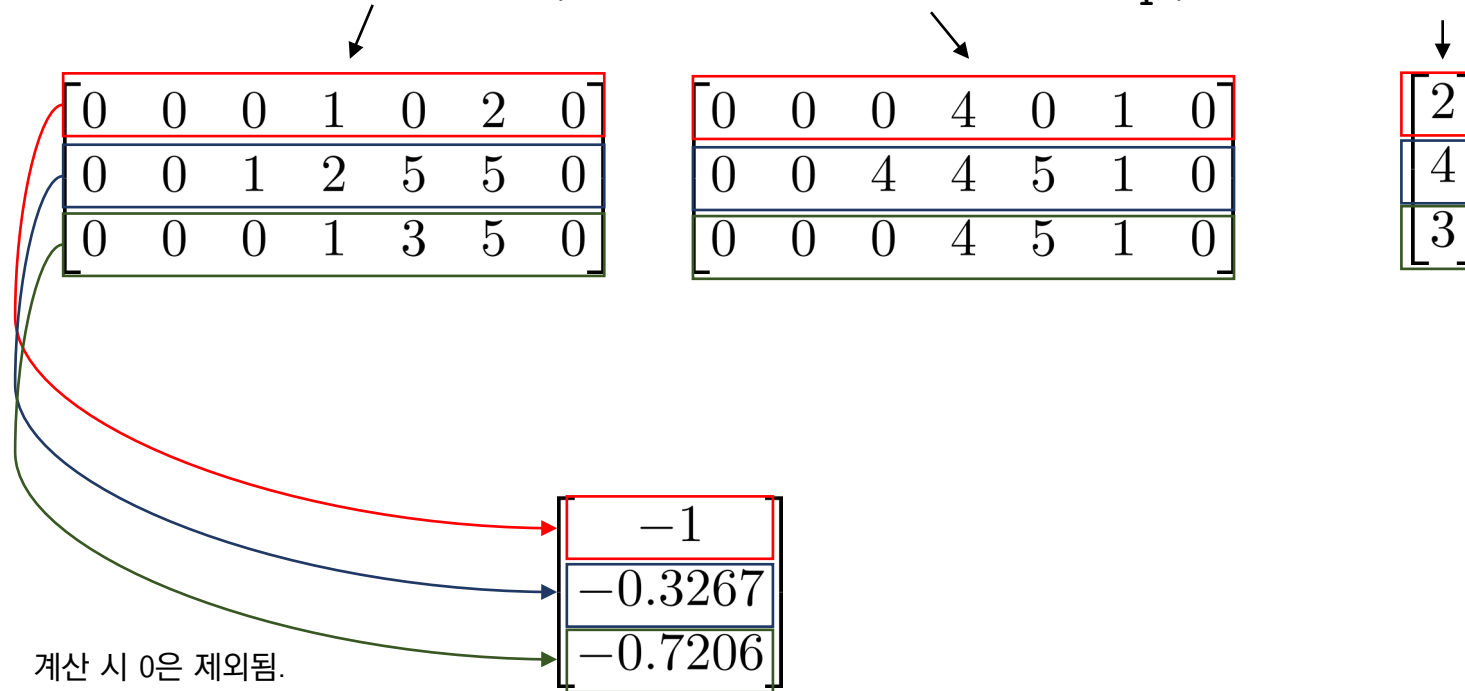
# Code 설명 – get\_cf\_feasibility

```
23      corr_array = corrc eof_sparse_mat_vec(this_user_mat, this_user_mat_overlap, this_user_vec)
```

23. this\_user\_mat과 this\_user\_mat\_overlap의 각 행벡터에 대해 correlation을 계산하고, 이를 corr\_array에 저장.

# Code 설명 – get\_cf\_feasibility

```
corrcoef_sparse_mat_vec(this_user_mat, this_user_mat_overlap, this_user_vec)
```



# Code 설명 – get\_cf\_feasibility

```
24 if std or export_mode:
    ret['RATING_STAT'] = {}
    ret['RATING_STAT']['THIS_RATING'] = (this_rating_mean, this_rating_std)
    if not export_mode:
        o_mean = np.asarray(mean_sparse_mat(this_user_mat, axis=1)).reshape(-1).tolist()
        o_std = np.asarray(np.sqrt(var_sparse_mat(this_user_mat, axis=1))).reshape(-1).tolist()
        ret['RATING_STAT']['OTHER_RATING'] = dict(zip(this_other_user_id_mat.tolist(), zip(o_mean, o_std)))

    if not export_mode:
        ret['OVERLAP_DATA'] = dict(zip(this_other_user_id_mat.tolist(), corr_array.tolist()))
    ret['SUMMARY_STAT']['AVG_OVERLAP'] = np.mean(this_user_vec)
    ret['SUMMARY_STAT']['STD_OVERLAP'] = np.std(this_user_vec)
    ret['SUMMARY_STAT']['MIN_OVERLAP'] = np.min(this_user_vec)
    ret['SUMMARY_STAT']['MAX_OVERLAP'] = np.max(this_user_vec)
    ret['SUMMARY_STAT']['NUM_CORR_USER'] = len(this_user_vec)
    ret['SUMMARY_STAT']['AVG_CORR'] = np.mean(corr_array)
    ret['SUMMARY_STAT']['STD_CORR'] = np.std(corr_array)
```

24. 관련 요약 통계량들을 반환 dictionary에 저장.

# Code 설명 – get\_cf\_feasibility

```
25     ret['FAIL'] = 1 if fail else 0
26     if fail:
27         ret['FAIL_REASON'] = cf_fail_reason
28
29     return ret
```

25. CF 추천 가능 여부 저장

26. CF 추천 불가능시 이유 저장

27. ret를 반환하여 함수 종료



# Code 설명

```
genre_cols = [col for col in ani_meta_df.columns if col.startswith('gnr_') == True]
def gnr_extract(x):
    for col in genre_cols:
        if x[col] == 1:
            return col.split('_')[1]

for idx, ani in ani_meta_df.iterrows():
    ani_meta_df.loc[idx, 'gnr'] = gnr_extract(ani)

all_anime = [(ani_id, ani_rating_df.columns.get_loc(ani_id) - 1) for ani_id in ani_rating_df.columns if ani_id != 'svc_mgmt_num']
```

1. 애니메이션 메타데이터('contents\_list\_2156.csv')에서 장르를 나타내는 컬럼 이름을 추출하여 리스트로 저장.
2. 'gnr'라는 새로운 컬럼을 만들고 해당 컬럼의 값은 장르를 나타내도록 지정.
3. 'ani\_rating.csv'의 각 컬럼으로부터 각 애니메이션 ID와 (컬럼 번호 - 1)을 순서쌍으로 얻어내고, 이를 all\_anime 리스트에 저장.

# Code 설명 - recommendation

```
1  if cut <= 0 or cut > 1:
    print('잘못된 Parameter 요청. cut은 반드시 0과 1 사이의 값이어야합니다.')
    return 'None'

2  if min_prop < 0 or min_prop > 1:
    print('잘못된 Parameter 요청. min_prop는 반드시 0 또는 0과 1사이의 값이어야합니다.')
    return 'None'

3  if (type(min_ir_user_num) == int and min_ir_user_num < 0) or (type(min_ir_user_num) != int):
    print('잘못된 Parameter 요청. min_ir_user_num은 반드시 양의 정수 값이어야합니다.')
    return 'None'
```

입력된 파라미터의 유효성 검사

1. cut: 0보다 크고 1보다 작은 숫자인지 확인
2. min\_prop: 0 또는 0과 1사이의 숫자인지 확인
3. min\_ir\_user\_num: 양의 정수인지 확인

# Code 설명 - recommendation

```
4     find_idx = sp.sparse.find(CS_mat == user_id)[1]
5     user_idx = find_idx.item() if len(find_idx) > 0 else -1
6     if user_idx == -1:
        print('추천 실패: 존재하지 않는 ID')
        return 'None'
```

4. 해당 유저 아이디에 대응하는 유저 - 평점 행렬의 행 인덱스 값을 찾음.

5 - 6. 찾을 수 없을 경우 user\_idx에 -1값 할당 및 안내 메시지 출력.

# Code 설명 - recommendation

```
7     this_CF_mat = CF_mat[user_idx]
8     wat_anime_list = []
9     for anime_id, anime_idx in all_anime:
        if this_CF_mat[0,anime_idx] > 0:
            wat_anime_list.append(anime_id)
```

7. 해당 유저의 평점 정보를 불러옴.

8 - 9. 해당 유저가 시청한 애니메이션 ID를 저장한 리스트를 생성.

# Code 설명 - recommendation

```
10  wat_genre = {}  
    for col in genre_cols:  
        if ani_meta_df.loc[wat_ani_list, col].sum() > 0:  
            wat_genre[col.split('_')[1]] = ani_meta_df.loc[wat_ani_list, col].sum() / ani_meta_df.loc[wat_ani_list, genre_cols].to_numpy().sum()
```

10. 해당 유저가 시청한 애니메이션들의 장르 및 장르별 비율을 (key, value) 형태로 dictionary에 저장.

# Code 설명 - recommendation

```
11 sorted_wat_genre = {genre: ratio for genre, ratio in sorted(wat_genre.items(), key=lambda item: item[1], reverse=True) if ratio > min_prop}
12 if len(sorted_wat_genre) == 0:
    if verbose:
        print('추천 실패: 가장 높은 시청 비율 ({0})이 min_prop 파라미터 값인 ({1})보다 작습니다. min_prop 파라미터가 지나치게 높게 설정되어 있습니다.'.format(min_prop, sorted_wat_genre.keys()[0]))
    return 'None'
```

11. 10에서 얻은 장르 dictionary를 value 값(장르 비율) 중에서 내림차순으로 정렬하며, min\_prop보다 비율이 작은 장르는 삭제함.

12. 만약 모든 장르의 비율이 min\_prop보다 작을 경우, 오류 메시지 출력 및 'None' 반환.

# Code 설명 - recommendation

```
13     dominate_genre = list({k: sorted_wat_genre[k] for k in list(sorted_wat_genre)[:find_opt_key_num(sorted_wat_genre, cut=cut)]}.keys())
14     if len(dominate_genre) == 0:
        print('추천 실패: 장르 선별 과정에서 모든 장르가 탈락되었습니다. cut 파라미터와 min_prop 파라미터 값을 적절히 조정해야 합니다.')
        return 'None'
```

13. 최소 비율을 넘긴 장르 중에서, 가장 많이 본 상위 장르를 추출하여 `dominate_genre` 리스트에 저장.

14. 만약 상위 장르가 없다면, 오류 메시지 출력 후 'None' 반환.

# Code 설명 - recommendation

```
15 recommend_candidates = [(ani_id, ani_idx) for ani_id, ani_idx in all_ani
                             if (ani_id not in wat_ani_list) and (ani_meta_df.loc[ani_id, 'gnr'] in dominate_genre)]
    ..
```

15. 상위 장르 중에서, 해당 유저가 시청하지 않은 애니메이션들에 대해 (애니메이션 ID, 애니메이션 칼럼 인덱스) 순서쌍을 `recommend_candidates` 리스트로 저장.



# Code 설명 - recommendation

```
16     cf_info = get_cf_feasibility(user_idx, std=std)
17     if cf_info['FAIL'] == 1:
18         if verbose:
19             print('추천 실패: Infeasible on CF (' + cf_info['FAIL_REASON'] + ')')
20         return 'None'
21     cf_cs = list(cf_info['OVERLAP_DATA'].keys())
22     cf_users_idx = np.where(np.isin(CS_mat, cf_cs))[0]
```

16. 해당 유저의 CF Feasibility 정보를 cf\_info로 불러옴.

17. CF 기반 추천이 불가능할 경우 안내 메시지 출력 후 'None' 값을 반환하고 종료.

18 - 19. CF 계산에 활용된 다른 유저들의 ID와 인덱스를 각각 cf\_cs 리스트와 cf\_user\_idx에 저장.

# Code 설명 - recommendation

```
fail = []
wat_anime_users = {}
for anime_id, anime_idx in recommend_candidates:
19     wat_users_idx = np.intersect1d(cf_users_idx, positive_rating_idx_dict[anime_idx])
20     wat_users = [user for user in CS_mat[wat_users_idx].tolist() if user != user_id]

21     if len(wat_users) > 0:
        fail.append(0)
        wat_users_rating = CF_mat[wat_users_idx, anime_idx].toarray().reshape(-1).tolist()
        wat_anime_users[anime_id] = dict(zip(wat_users, wat_users_rating))
    else:
        fail.append(1)
```

19 - 20. 추천 대상 애니메이션마다, 해당 애니메이션을 시청하고, CF 계산에 활용된 다른 유저들의 인덱스 값 및 ID를 각각 wat\_user\_idx와 wat\_users에 저장.

21. 그러한 유저들이 적어도 한 명 존재한다면, fail 리스트에 0을 추가하고, 그 유저들의 rating을 wat\_users\_rating에 저장. 최종적으로 wat\_anime\_users에 해당 애니메이션에 대하여, 그 유저들의 ID와 rating들을 dictionary 형태로 저장. 만약 해당 애니메이션에 대해 조건을 만족하는 유저가 없다면 fail 리스트에 1을 추가.

```
22  if sum(fail) == len(recommend_candidates):  
    if verbose:  
        print('추천 실패: Correlation 계산에 포함된 다른 유저들 중, 추천 후보 애니메이션을 시청한 유저가 존재하지 않음.')    return 'None'
```

22. 만약 모든 추천 후보 애니메이션에서, 그 애니메이션을 시청하고 동시에 CF 계산에 이용된 유저들이 존재하지 않는다면, 오류 메시지를 출력하고, 'None'을 반환함.

# Code 설명 – recommendation

```
23 for ani_id, wat_rating_info in wat_anis_users.items():
    user_id_list = list(wat_rating_info.keys())
24     if len(user_id_list) < min_ir_user_num:
        fail.append(1)
        continue
    else:
        fail.append(0)
    corrcoeffs = np.array([cf_info['OVERLAP_DATA'][id] for id in user_id_list])
    if std:
        explicit_ratings = np.array([(wat_rating_info[id] - cf_info['RATING_STAT']['OTHER_RATING'][id][0]) / cf_info['RATING_STAT']['OTHER_RATING']
        if np.sum(np.abs(corrcoeffs)) == 0: # 모든 Weight들이 0
            this_ir = 0
        else:
25             this_ir = np.sum(corrcoeffs * explicit_ratings) / np.sum(np.abs(corrcoeffs))
            this_ir = this_ir * cf_info['RATING_STAT']['THIS_RATING'][1] + cf_info['RATING_STAT']['THIS_RATING'][0]
        else:
            explicit_ratings = np.array([wat_rating_info[id] for id in user_id_list])
            this_ir = np.sum(corrcoeffs * explicit_ratings) / np.sum(np.abs(corrcoeffs))
    ratings[ani_id] = this_ir
```

23. 추천대상이 되는 각각의 애니메이션에 대해서, CF 계산을 할 유저 ID 리스트를 `user_id_list`에 저장 .

24. 만약 유저 개수가 기준 개수보다 부족하면, 해당 애니메이션은 고려하지 않고 다음으로 넘어감.

25. 23에서의 유저들과 해당 유저의 correlation을 `corrcoeffs` 리스트에 저장 후, correlation을 weight로 하여 해당 애니메이션의 expected rating을 계산.

```
26  if sum(fail) == len(wat_ani_users):  
    if verbose:  
        print('추천 실패: 추천 후보 애니메이션을 시청한 유저가 충분히 존재하지 않음.')  
    return 'None'
```

26. 만약 모든 추천 후보 애니메이션에서, CF 계산에 고려되는 다른 유저들의 수가 기준 수를 넘지 못하면, 오류 메시지를 출력하고, 'None'을 반환함.

```
27 sorted_ratings = {ani_id: rating for ani_id, rating in sorted(ratings.items(), key=lambda item: item[1], reverse=True)}
```

27. 최종 추천 후보 애니메이션을 expected rating 순으로 정렬함.

# Code 설명 - recommendation

```
28  if output_len > len(sorted_ratings):  
    if verbose:  
        print('출력하고자 하는 애니메이션의 개수가 최종 후보 애니메이션의 개수보다 크기 때문에, 총 {0}개의 추천  
        output_len = len(sorted_ratings)  
  
29  output_dict = {k: sorted_ratings[k] for k in list(sorted_ratings)[:output_len]}  
    if verbose:  
        print('출력되는 추천 애니메이션의 이름과 그 기대 평점은 아래와 같습니다.')  
        for (ani_id, rating) in output_dict.items():  
            print(ani_meta_df.loc[ani_meta_df.index == ani_id]['sris_nm'].values[0] + ': ' + str(rating))  
    if output_len == 1:  
        return list(output_dict.keys())[0]  
    return ", ".join(list(output_dict.keys()))
```

28. 원하는 추천 애니메이션 출력 수가 가능한 출력 수보다 클 경우 가능한 출력수로 설정

29. ratings dictionary에서 출력할 dictionary 값을 추출 후, 원하는 추천 애니메이션 출력 수에 맞게 최종 결과 반환